

*Doubly Linked with Class –
Destroy List[Tweak Only]*

At case 12: when we call →

```
case 12:
    list.destroyList();
    break;
```

Then :

```
private:
    // Helper flag to avoid double messages when destructor
    is called
    bool isDestroying = false;

    // This method is called by the destructor
    void prepareForDestruction()
    {
        isDestroying = true;
    }
};
```

→ isDestroying remain false .

```
void destroyList()
{
    if (head == nullptr )
    {
        // Only print a message if it's a user action.
        if (!isDestroying)
        {
            // isDestroying flag prevents double "List
            is empty" message from destructor
            cout << "List is already empty." << endl;
        }
    }
}
```

```

        return;
    }
.....
    if (!isDestroying){
        cout << "List destroyed." << endl;
    }
}

```

Here , !isDestroying means isDestroying = False ∴

it will first check head = NULL , if the condition is true , then:

! isDestroying [isDestroying = false] is True, then:

print message: "List is already empty";

Retrun → Exit from the function.

If List is not empty, then , List will get emptied and head will get assigned to

NULL and as ! isDestroying [isDestroying = false] is True, then:

print message: "List destroyed";

→ This is Manual Destruction of List.

Automatic Destruction

But when the Program exits , the driver main function finishes , the list object goes out of scope.

```

DoublyLinkedList list; -> Goes out of scope as Program
exits.

```

C++ automatically calls its destructor (~DoublyLinkedList()) to prevent memory leaks. This is a background clean up operation and Printing "List destroyed." at this point will be redundant and confusing for the user hence,

```
~DoublyLinkedList()
{
    prepareForDestruction();
    destroyList();
}
```

Now, prepareForDestruction() ; gets executed.

```
private:
    // Helper flag to avoid double messages when destructor
    is called
    bool isDestroying = false;

    // This method is called by the destructor
    void prepareForDestruction()
    {
        isDestroying = true;
    }
};
```

isDestroying will be set to true ,hence,

```
void destroyList()
{
    if (head == nullptr )
    {
        // Only print a message if it's a user action.
        if (!isDestroying)
        {
```

```

        // isDestroying flag prevents double "List
is empty" message from destructor
        cout << "List is already empty." << endl;
    }
    return;
}
.....
    if (!isDestroying){
        cout << "List destroyed." << endl;
    }
}

```

it will first check head = NULL ,if the condition is true ,then:

*!isDestroying [isDestroying = false]is False, the inner
if statement doesn't get executed.*

Retrun → Exit from the function.

if List is not Empty or Head ≠ NULL then:

```

void destroyList()
{
    if (head == nullptr ){....}

    Node *current = head;
    Node *nextNode;
    while (current != nullptr)
    {
        nextNode = current->next;
        delete current;
        current = nextNode;
    }
    head = nullptr;
    if (!isDestroying){....}
}

```

Line of Codes Gets Executed:

```
Node *current = head;
    Node *nextNode;
    while (current != nullptr)
    {
        nextNode = current->next;
        delete current;
        current = nextNode;
    }
    head = nullptr;
```

And then again:

```
    if (!isDestroying){
        cout << "List destroyed." << endl;
    }
```

if(! isDestroying){...}is aborted as :

! isDestroying [isDestroying = false]is False.

i. e. silently destroying the List without printing any message to user.
